CampusConnect

Plateforme de vie étudiante — événements universitaires, inscriptions, avis et publication par les clubs

Type de document	Rapport synthétique de projet
Projet	CampusConnect (application web MERN)
Technologies	MongoDB · Express · React · Node.js
Réalisé par	_____
Encadré par	_____
Année universitaire	2025 – 2026

Sommaire

1. Présentation du projet
2. Architecture technique
3. Schémas Mongoose (modèle de données)
4. API REST
5. Captures d'écran de l'application
6. Conclusion et perspectives

1. Présentation du projet

CampusConnect est une application web qui centralise la vie associative et événementielle d'un campus universitaire. Elle permet aux étudiants de découvrir les événements, de s'y inscrire et de laisser un avis, et aux clubs de publier leurs propres événements sous le contrôle d'un administrateur.

Objectifs

- Regrouper en un seul endroit tous les événements du campus (culture, sport, technologie, carrière, science).
- Simplifier les inscriptions et le suivi des places disponibles.
- Permettre la publication d'événements par les clubs avec une validation par un administrateur.
- Recueillir la satisfaction des participants via des avis notés (1 à 5 étoiles).

Acteurs et rôles

Rôle	Droits et fonctionnalités
étudiant	Consulter les événements, s'inscrire / se désinscrire, gérer « Mes inscriptions », donner un avis (après inscription).
club	Toutes les actions étudiant + publier de nouveaux événements (soumis à validation) et gérer ses publications.
admin	Valider ou refuser les événements en attente, publier directement, supprimer tout événement ou avis.
visiteur	Parcourir les événements en lecture seule ; invitation à se connecter pour s'inscrire.

Fonctionnalités principales

- **Authentification** par e-mail / mot de passe avec jeton JWT et mot de passe haché (bcrypt).
- **Tableau de bord** : statistiques personnelles, compte à rebours du prochain événement.
- **Exploration** des événements : recherche plein texte, filtre par catégorie, tri, places disponibles, événements passés.
- **Inscriptions** en un clic avec suivi de la capacité (barre de remplissage).
- **Avis** : note + commentaire, un seul avis par étudiant et par événement.
- **Publication** par les clubs et **modération** par l'administrateur.

2. Architecture technique

L'application suit une architecture **client / serveur** classique : une interface React (SPA) consomme une API REST Node/Express, qui persiste les données dans MongoDB via Mongoose.

Couche	Technologies
Front-end	React 18, Vite 5, React Router 6, Axios, CSS personnalisé (thème clair/sombre)
Back-end	Node.js, Express 4, Mongoose 8, JSON Web Token, bcrypt, CORS, dotenv
Base de données	MongoDB (hébergement MongoDB Atlas)
Déploiement	Front : Vercel · API : Render · Base : MongoDB Atlas

Arborescence du projet

```
campusconnect/
├── server/                 # API REST (Node.js / Express)
│   ├── models/             # Schémas Mongoose : User, Event, Feedback
│   ├── routes/             # auth.js, events.js, feedback.js
│   ├── middleware/         # protect.js (JWT + contrôle des rôles)
│   ├── seed.js             # Jeu de données de démonstration
│   └── server.js           # Point d'entrée de l'API
└── client/                 # Interface utilisateur (React / Vite)
    └── src/
        ├── components/     # Layout, EventCard, Stars
        ├── pages/          # Dashboard, Browse, EventDetail, Publish…
        ├── AuthContext.jsx # Contexte d'authentification
        └── api.js          # Client Axios (injection du JWT)
```

Flux d'une requête

Le navigateur envoie une requête HTTP à l'API. Si la route est protégée, le middleware `protect` vérifie l'en-tête `Authorization: Bearer <token>`, décode le JWT, charge l'utilisateur puis laisse passer. Un second middleware, `restrictTo( … )`, limite l'accès à certains rôles. La réponse est renvoyée au format JSON.

> **Sécurité :** mots de passe hachés avec **bcrypt** (coût 10), jetons **JWT** valables 7 jours, accès CORS restreint aux origines autorisées, contrôle systématique des rôles côté serveur.

3. Schémas Mongoose (modèle de données)

Trois modèles composent la base : **User**, **Event** et **Feedback**.

3.1 User

```
const userSchema = new mongoose.Schema({
  nom:        { type: String, required: true, trim: true },
  email:      { type: String, required: true, unique: true, lowercase: true, trim: true },
  motDePasse: { type: String, required: true, minlength: 6 },
  role:       { type: String, enum: ['etudiant','club','admin'], default: 'etudiant' },
}, { timestamps: true });
```

Champ	Type	Contraintes
`nom`	String	Obligatoire, espaces retirés
`email`	String	Obligatoire, **unique**, minuscules
`motDePasse`	String	Obligatoire, 6 caractères minimum (haché)
`role`	String	Valeurs : `etudiant` · `club` · `admin`

3.2 Event

```
const eventSchema = new mongoose.Schema({
  titre:        { type: String, required: true, trim: true },
  description:  { type: String, required: true },
  categorie:    { type: String, enum: ['Technologie','Culture','Sport','Carrière','Science'], required: true },
  niveau:       { type: String, enum: ['Tous niveaux','Débutant','Intermédiaire','Avancé'], default: 'Tous niveaux' },
  date:         { type: Date, required: true },
  heureDebut:   { type: String, required: true },
  heureFin:     { type: String, required: true },
  lieu:         { type: String, required: true },
  capacite:     { type: Number, required: true, min: 1 },
  tags:         [String],
  statut:       { type: String, enum: ['en_attente','publie'], default: 'en_attente' },
  organisateur: { type: ObjectId, ref: 'User', required: true },   // User 1 → N Event
  inscrits:     [{ type: ObjectId, ref: 'User' }],                 // User N ⇔ N Event
}, { timestamps: true });
```

Champ	Type	Contraintes
`titre`	String	Obligatoire
`description`	String	Obligatoire
`categorie`	String	Obligatoire · 5 catégories (enum)
`niveau`	String	Défaut « Tous niveaux »
`date` · `heureDebut` · `heureFin`	Date / String	Obligatoires
`lieu`	String	Obligatoire
`capacite`	Number	Obligatoire, minimum 1
`tags`	[String]	Liste libre (mots-clés)
`statut`	String	`en_attente` · `publie`
`organisateur`	ObjectId → User	Obligatoire (référence)
`inscrits`	[ObjectId → User]	Liste des participants

3.3 Feedback

```
const feedbackSchema = new mongoose.Schema({
  event:       { type: ObjectId, ref: 'Event', required: true },  // Event 1 → N Feedback
  auteur:      { type: ObjectId, ref: 'User',  required: true },
  note:        { type: Number, required: true, min: 1, max: 5 },
  commentaire: { type: String, required: true, maxlength: 500 },
}, { timestamps: true });
feedbackSchema.index({ event: 1, auteur: 1 }, { unique: true });  // 1 avis par étudiant et par événement
```

Champ	Type	Contraintes
`event`	ObjectId → Event	Obligatoire
`auteur`	ObjectId → User	Obligatoire
`note`	Number	Entre 1 et 5
`commentaire`	String	Obligatoire, 500 caractères max.
Index unique sur (`event` , `auteur`) → un étudiant ne peut laisser qu'un seul avis par événement.		

3.4 Relations entre les modèles

User → Event
1 → N
Un organisateur (club/admin) crée plusieurs événements via `organisateur`.

Event ↔ User
N ↔ N
Un événement a plusieurs inscrits ; un utilisateur s'inscrit à plusieurs événements (`inscrits[]`).

Event → Feedback
1 → N
Un événement reçoit plusieurs avis (`Feedback.event`).

User → Feedback
1 → N
Un utilisateur écrit plusieurs avis (`Feedback.auteur`).

Toutes les références sont résolues côté API avec `.populate()`. La note moyenne et le nombre d'avis d'un événement sont calculés par une agrégation MongoDB (`$group`/`$avg`).

4. API REST

Points d'entrée exposés par le serveur Express (préfixe `/api`).

Méthode	Route	Accès	Description
POST	`/auth/register`	Public	Créer un compte (étudiant ou club)
POST	`/auth/login`	Public	Connexion, renvoie un jeton JWT
GET	`/auth/me`	Connecté	Profil de l'utilisateur courant
GET	`/events`	Public	Liste des événements publiés (recherche, filtre, tri)
GET	`/events/:id`	Public	Détail d'un événement
GET	`/events/mes-inscriptions`	Connecté	Événements auxquels l'utilisateur est inscrit
GET	`/events/en-attente`	Admin	Événements à valider
POST	`/events`	Club, Admin	Créer un événement
PATCH	`/events/:id/valider`	Admin	Valider (publier) un événement
POST	`/events/:id/inscription`	Connecté	S'inscrire à un événement
DELETE	`/events/:id/inscription`	Connecté	Se désinscrire
DELETE	`/events/:id`	Organisateur, Admin	Supprimer un événement
GET	`/events/:id/feedback`	Public	Avis d'un événement
POST	`/events/:id/feedback`	Inscrit	Publier un avis
DELETE	`/feedback/:id`	Auteur, Admin	Supprimer un avis

> **Comptes de démonstration** (créés par `npm run seed`) :
> `admin@campus.tn` · `club@campus.tn` · `etudiant@campus.tn` — mot de passe : `password123`

5. Captures d’écran de l’application

Interface moderne à thème clair/sombre, captures réalisées sur l’application en fonctionnement.

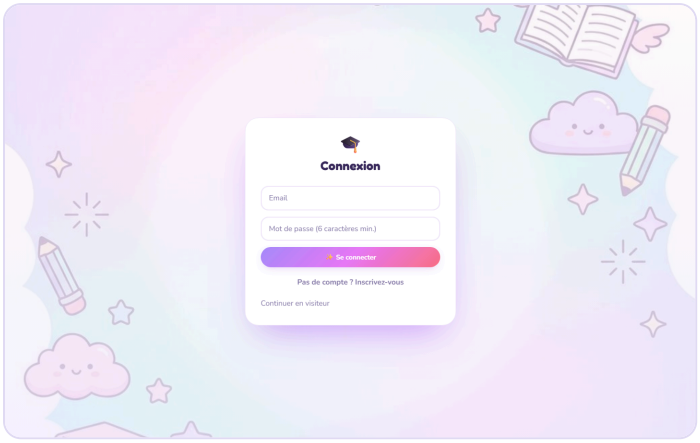


Figure 1 — Page de connexion / création de compte

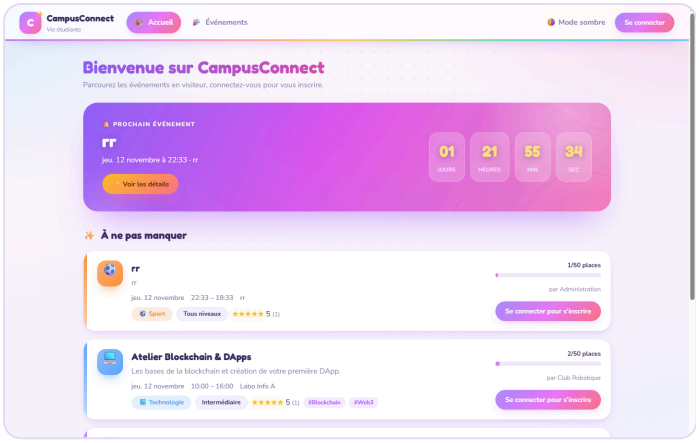


Figure 2 — Tableau de bord : événement à la une et compte à rebours

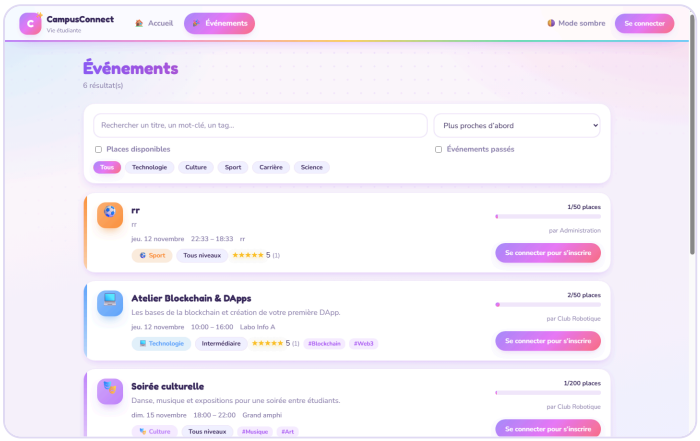


Figure 3 — Exploration des événements : recherche, filtres et tri

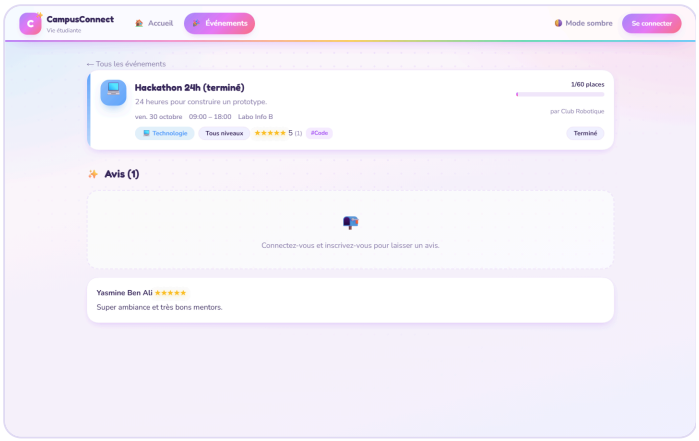


Figure 4 — Détail d’un événement : inscription, places et avis

6. Conclusion et perspectives

CampusConnect couvre l’ensemble du cycle de vie d’un événement universitaire : publication par les clubs, validation par l’administrateur, consultation publique, inscription et retour d’expérience via les avis. L’application repose sur une pile MERN cohérente, avec une API REST sécurisée (JWT, rôles, bcrypt) et une interface réactive offrant un thème clair et sombre.

Pistes d’évolution :

- Notifications par e-mail (confirmation d’inscription, rappel d’événement).
- Export du calendrier (iCal) et gestion des listes d’attente.
- Tableau de bord statistique pour les clubs (taux de remplissage, satisfaction).
- Authentification via le compte universitaire (SSO) et modération plus fine des avis.